

SE 811: Software Maintenance

Toukir Ahammed

Preventive Change

- Preventive change is the change applied to prevent malfunctions or to improve the maintainability of software.
- As a software system is enhanced, modified, and adapted to new requirements, the code becomes more complex and drifts away from its original design, thereby lowering the quality of the software.
- According to Lehman's laws of software evolution, the long-term effect of corrective, adaptive, and perfective changes is deteriorating the software structure, while increasing entropy.
- Preventive changes are usually applied to address these problems.

Preventive Change

- For instance, after developers fix some bugs and implement new features in an existing software product, the complexity of source code can increase to an unmanageable level.
- Through code *refactoring* developers can reduce code complexity and increase the readability, reusability, and maintainability of software.
- *Refactoring* copes with increasing software complexity by transforming a program from one representation to another while preserving the program's external behavior (functionality and semantics).

Refactoring Is Not Easy!

Refactoring process consists of the following activities:

1. Identifying where to apply what refactoring(s).
2. Checking that the refactoring to apply preserves program behaviors.
3. Refactoring the code.
4. Assessing the effect of applied refactoring on software quality (e.g., complexity and readability).
5. Maintaining the consistency between refactored code and other related software artifacts, like documentation, tests, and issue tracking records.

High-level vs Low-level Refactorings

- High-level refactorings (i.e., composite refactorings) reflect more complex behavior-preserving transformations.
 - creating an abstract superclass,
 - subclassing and simplifying conditionals,
 - capturing aggregations and components, etc.
- Low-level refactorings are primitive operations such as creating, deleting, or changing a program entity.

Does Refactoring Always Preserve Behavior?

- While refactoring is defined as behavior-preserving code transformations in the academic literature, the de facto definition of refactoring in practice seems to be very different from such rigorous definition.
- Martin Fowler catalogs 72 types of structural changes in object-oriented programs, but these transformations do not necessarily guarantee behavior preservation.
- In fact, Fowler recommends developers to write test code first, since these refactorings may change a program's behavior.

Does Refactoring Always Preserve Behavior?

- Murphy-Hill et al. analyzed refactoring logs and found that developers often interleave refactorings with other behavior-modifying transformations, indicating that pure refactoring revisions are rare.
- Johnson's refactoring definition is aligned with these findings—

“refactoring improves behavior in some aspects but does not necessarily preserve behavior in all aspects”

Systematic Refactoring

- Refactoring operations are systematic in the sense that they involve repetitive nonlocal transformations.
- Systematic refactoring contain multiple similar transformations at a code level.
- For example, creating an abstract superclass involves moving multiple variables and functions common to more than one sibling classes to their common superclass.
- Subclassing and simplifying conditionals consist of creating several classes, each of which is in charge of evaluating a different conditional.
- Capturing aggregations and components usually involves moving multiple members from a component to an aggregate object.

Contradicting Beliefs on Refactoring Benefits

On one hand, some believe that refactoring improves software quality and maintainability and a lack of refactoring incurs *technical debt* to be repaid in the future in terms of increased maintenance cost.

On the other hand, some believe that refactoring does not provide immediate benefits unlike bug fixes and new features during software evolution.

Support for the Benefits of Refactoring

- Researchers found empirical evidence that bug fix time decreases after refactoring and defect density decreases after refactoring.
- More specifically, it is found that the productivity measure manifested by the average time taken to resolve tickets decreases after re-architecting the system.
- In defect prediction models based on software evolution attributes, it is found that refactoring-related features and defects have an inverse correlation —*if the number of refactorings increases in the preceding time period, the number of defects decreases.*

Support for the Opposite View

- Refactoring may even incur additional bugs, researchers found that code churns are correlated with defect density and that refactorings are correlated with bugs.
- Changes involved only a single line of code has less chance to result in error, while large commits (changes which tend to include refactorings), have a higher chance of inducing bugs.
- Refactorings often occur together with other types of changes and that refactorings are followed by an increasing number of bugs.
- Researchers found that the number of bug fixes increases after API refactorings by investigating the relationship between API refactorings and bug fixes

Reasons Refactoring May Not Always Be Beneficial

- One reason why refactoring could be potentially error-prone is that refactoring often requires coordinated edits across different parts of a system, which could be difficult for programmers to locate all relevant locations and apply coordinated edits consistently.
- Errors can also be caused by incomplete refactorings
- Code churn (the number of added, deleted, and modified lines of code) is found to be correlated with defect density; since refactoring often introduces a large amount of structural changes to the system, some question the benefit of refactoring.

Automated Refactoring

- Modern IDE provides automatic support for a variety of refactorings, including *rename*, *move*, *extract method*, etc.
- With such support, developers do not need to worry about how to check for preconditions or postconditions before manually applying a certain refactoring.
- Instead, they can simply select the refactoring command from a menu (e.g., *extract method*) and provide necessary information to accomplish the refactoring (e.g., *the name of a new method*).
- The IDE's refactoring engine takes care of the precondition check, program transformation, and postcondition check.

Automated Refactoring Reduces Developer Effort

- For instance, when conducting a *create_method_function* refactoring, before inserting a member function F to a class C , developers should check for these preconditions:
 - (1) The function is not already defined locally.
 - (2) The signature matches that of any inherited function with the same name.
 - (3) The signature of corresponding functions in subclasses matches it.
 - (4) If there is an inherited function with the same name, either the inherited function is not referenced on instances of C and its subclasses, or the new function is semantically equivalent to the function it replaces.
 - (5) F will compile as a member of C .
- If any precondition is not satisfied, the refactoring should not be applied to the program.

Real-World Refactoring Practices

Several studies investigated refactoring practices in industry and also examined the current challenges and risks associated with refactoring. Here are some interesting findings:

- Most of the developers carry out refactorings in the context of bug fixes or feature additions, and these changes are generally not semantics-preserving.
- Most of refactoring is done manually as opposed to using automated refactoring tools.
- Programmers do not use automated refactoring despite their awareness of the availability of automated refactorings.

Real-World Refactoring Practices

- One reason was that developers started a refactoring manually but only partway through realized that the change was a refactoring that the IDE offered—by then, it was too late.
- Another complaint was that refactoring tools disrupted their workflow, forcing them to use a tool when they wanted to focus on code.
- Risks involved in refactorings: developers reported regression bugs, code churn, merge conflicts, time taken from other tasks, the difficulty of doing code reviews after refactoring, and the risk of overengineering.